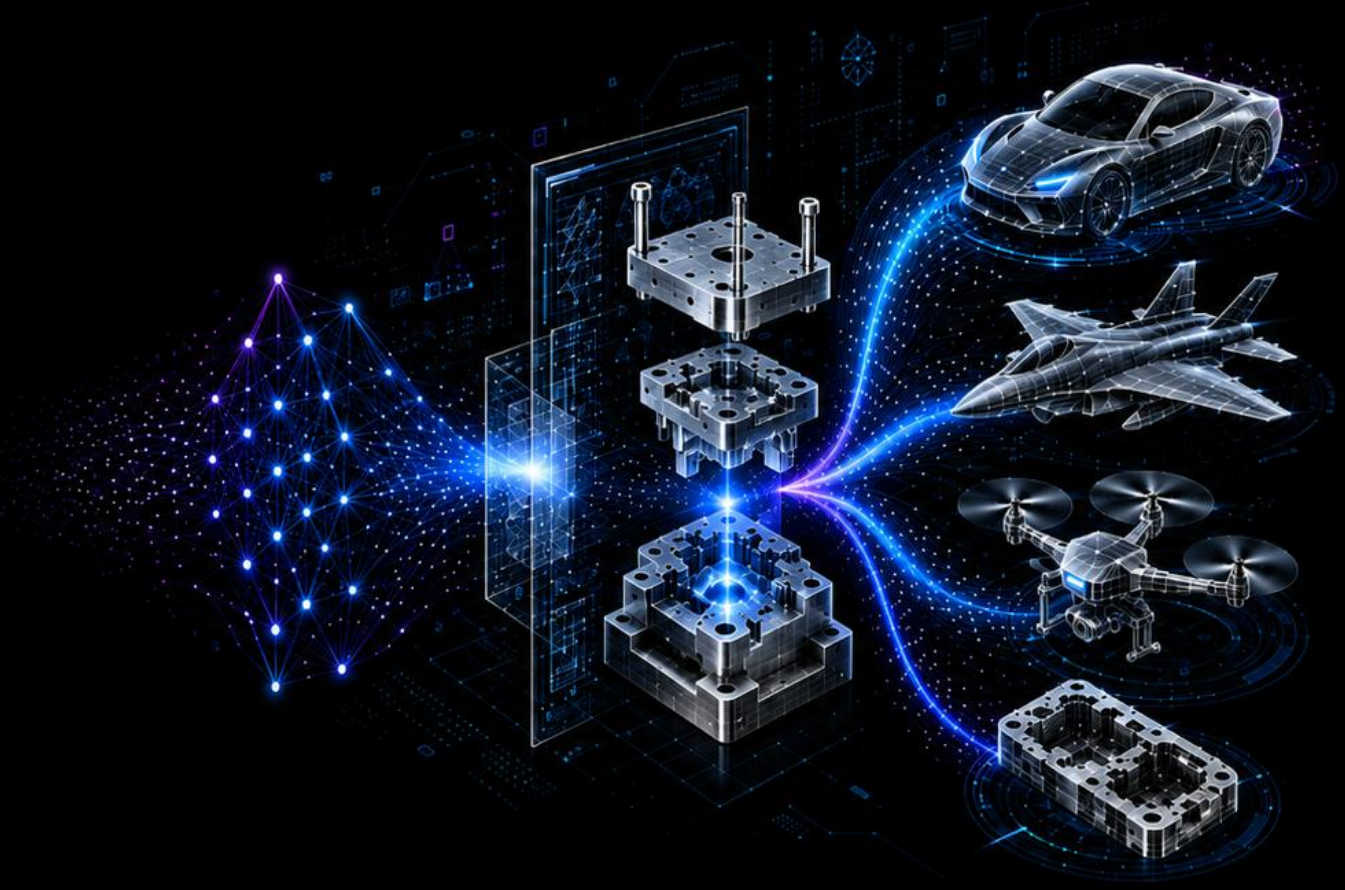
Meeting

Application method of RAG to LLM



신동우

(지도교수: 양선웅)

한양대학교 ERICA 기계공학과

Intelligent Design for Engineering Applications Lab

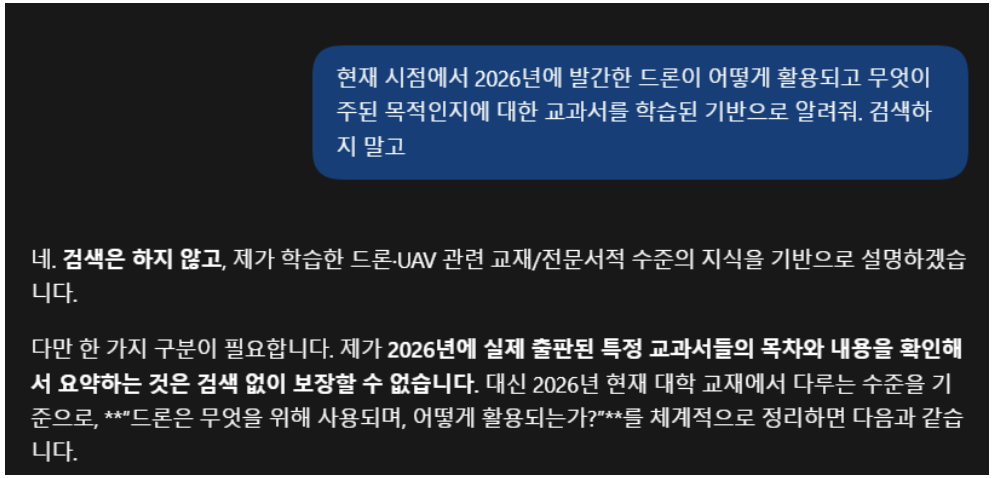
Application method of RAG to LLM

LLM의 한계 및 극복할 수 있는 방법

- LLM은 특정 시점까지의 데이터만 학습
- LLM의 한계 극복 방법

❖ 세세한 모든 것을 학습하지 않기 때문에 외부 검색 필요

❖ Local LLM을 활용하면 외부 지식 검색을 활용할 수 없다



방대한 텍스트를 읽고 기본 학습을 마친 LLM이 사용자의 구체적인 지시나 명령을 잘 따르는 '비서' 역할을 하도록 추가 훈련시키는 과정

❖ 첫 번째 방법: Fine-tuning or Instruction tuning
- ✓ 새 데이터로 모델 재 학습
- ✓ 모델의 파라미터가 업데이트

❖ 두 번째 방법: Naïve
- ✓ 원래 prompt와 함께 참고 문서 전체를 prompt에 넣기
- ✓ 예시: Drone 교과서 목차부터 끝 페이지까지

❖ 세 번째 방법: RAG(Retrieval Augmented Generation)
- ✓ Retrieval (검색): 외부 문서에서 관련 내용을 검색
- ✓ Augmented (보강): 그 내용으로 prompt를 보강
- ✓ Generation (생성): LLM이 보강된 prompt로 답변 생성

Application method of RAG to LLM

RAG

- RAG의 기본 구동 방식
- Embedding & Chunk

Indexing (색인): 문서를 불러와 분할-〉 Embed -〉 벡터DB저장
Retrieval (검색): 사용자 질문과 의미적으로 유사한 문서 조각 검색
Augmentation (증강): 질문 + 검색된 문서를 하나의 프롬프트로 결합
Generation (생성): LLM이 제공된 문서 범위 안에서 답변 생성

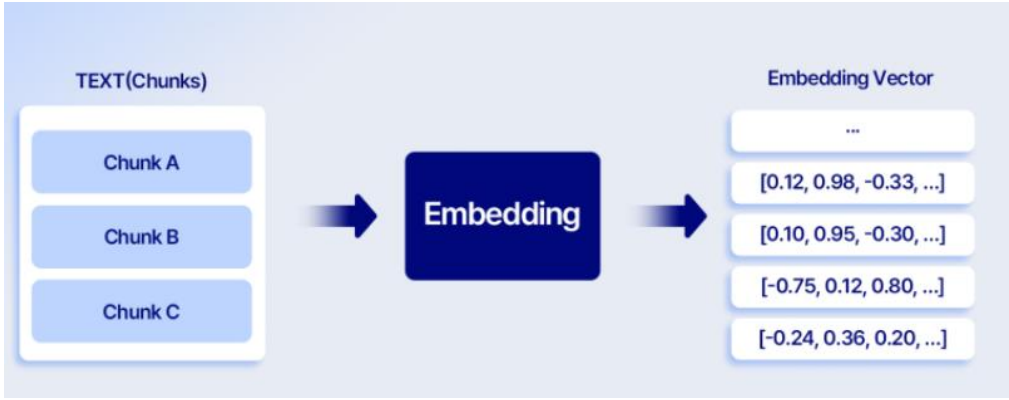
1. 사용자가 질문 입력
2. Embedding Model이 질문을 Vector로 변환
3. Vector DB에서 유사한 문서 Chunk Top-K/P 검색
4. [질문 + 검색된 문서]를 합쳐서 prompt 구성
5. LLM이 최종 답변 생성

❖ Embedding
- ✓ Text(문장)를 고차원 숫자 Vector로 변환
- ✓ 변환된 Vector를 기준으로 검색 수행
- ✓ 의미가 비슷한 문장 = Vector Space에서 가까운 위치

"독일로 출장을 갈 때 숙박비 상한선"

[0.23, 0.87, 0.11, 0.45, 0.99]

❖ Chunk
- ✓ 한 문장씩 또는 한 문단씩 잘라서 만들어진 덩어리



Application method of RAG to LLM

RAG

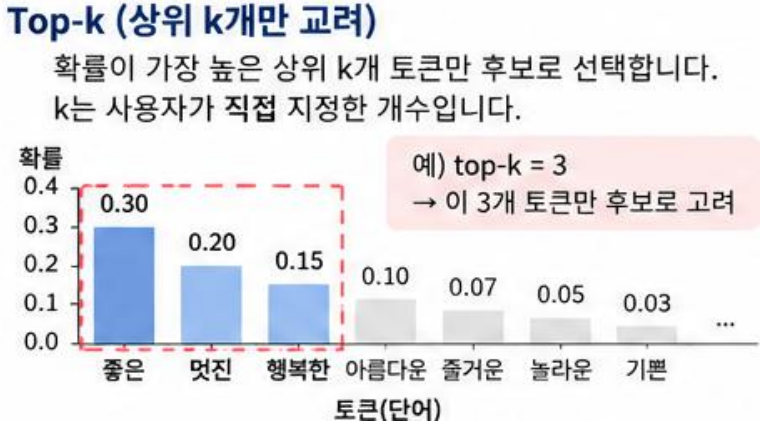
- Vector Similarity
- Concept of Top-K/P

❖ Similarity(유사도)

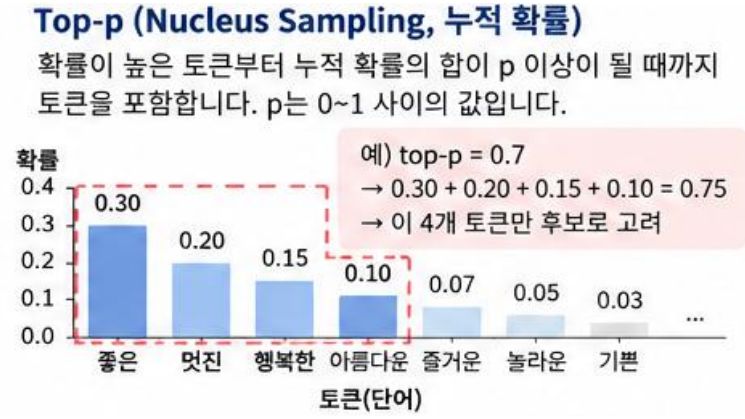
✓ Vector DB 중 현재 질문 Vector와의 Similarity를 계산



❖ Top-K



❖ Top-P



Application method of RAG to LLM

RAG

- Chunk가 문서의 문장을 어떻게 나누는가

Chunk

❖ RAG에서 Chunk는 원본 문서를 검색하기 좋은 크기로 잘라놓은 하나의 텍스트 조각

➔ RAG 뿐만 아니라 Fine tuning에서도 token 제한을 넘지 않도록 Chunk 단위로 학습하는데 활용

❖ 왜 문서를 굳이 Chunk로 나누는가

➔ 가장 중요한 이유는 하나의 문서에 여러 주제가 들어 있기 때문

➔ ex) 논문 전체를 vector 하나로 만든다고 하면 "FAISS에서 cosine similarity는 어떻게 되는가?" 라는 질문을 하면 하나의 vector로 압축되어 있기 때문에 FAISS 부분의 의미가 다른 내용에 묻힐 수 있다.

Chunk Size와 Overlap

❖ Chunk Size = 10 tokens, Overlap = 3tokens이라고 하면

The drone uses a lithium battery to supply power to the
[1] [2] [3] [4] [5] [6] [7] [8] [9] [10][11]
flight controller during autonomous flight operation
[12] [13] [14] [15] [16] [17]

단, 실제 tokenizer에서 한 단어가 반드시 1token은 아니다. 편의상 한 단어를 1token이라고 가정

Overlap

The drone uses a lithium battery to supply power to the
[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11]

flight controller during autonomous flight operation
[12] [13] [14] [15] [16] [17]

Next start = Chunk Size – Overlap
➔ 7token씩 이동하면서 10 token Chunk를 만든다고 생각하면 된다.

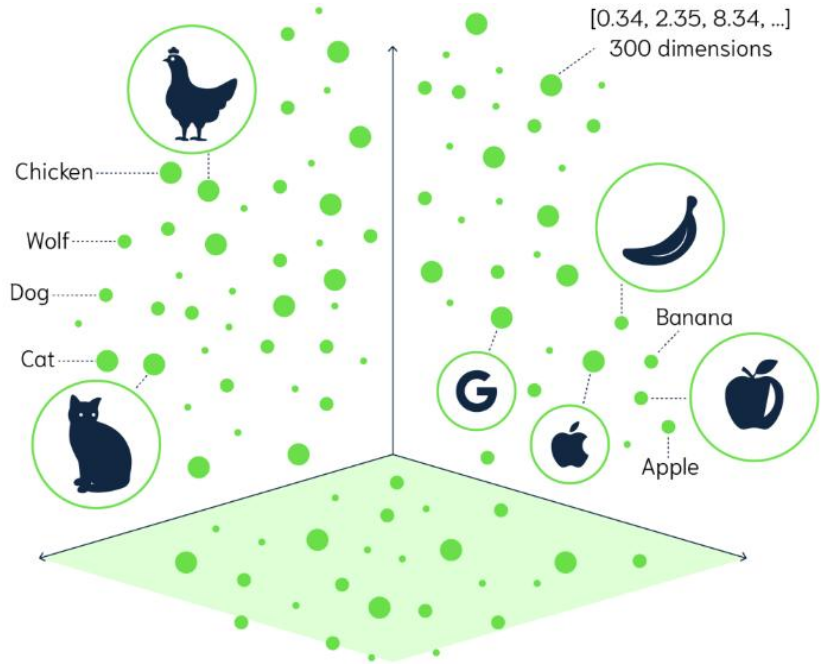
※ Overlap이 있다고 해서 문장의 의미적 경계에서 정확하게 잘리는 것은 아니다.

Application method of RAG to LLM

RAG(FAISS의 의미와 활용)

- FAISS의 의미
- FAISS의 활용

❖ FAISS(Facebook AI Similarity Search): Facebook(META)에서 개발하고 dense vector를 빠르게 검색하기 위한 라이브러리



-〉 Vector Space

❖ Example

Chunk 0:
"The MQ-9 Reaper is an unmanned aerial vehicle…"

Chunk1:
"Australia operates several types of military drones…"

Chunk2:
"New Zealand's active inventory includes…"

문장을 숫자 vector로 변환

Chunk 0 -〉 [0.12, -0.35, 0.81, …]
Chunk 1 -〉 [0.42, 0.11, 0.23, …]
Chunk 2 -〉 [0.12, -0.08, 0.64, …]

Q: What drones are listed in New Zealand's active inventory?

문장을 숫자 vector로 변환

Q: [0.68, -0.12, 0.59, …]

FAISS 활용으로 Question vector와 chunk vector 유사도 판단

Application method of RAG to LLM

RAG(FAISS의 의미와 활용)

- Embedding의 이해와 dimension

❖ Chunk A와 Chunk B가 있다고 가정하면

Chunk A: “The drone files fast.”
➔ Token 5개로 가정

Chunk B: “The military drones is operated by Australia.”
➔ Token 8개로 가정

각각의 토큰 수는 다르다. 하지만 Embedding model에선
고정된 d차원 vector로 표현

예를 들어서 d = 384라고 하면:
Chunk A –〉 5 x 384　　Chunk B –〉 8 x 384

Chunk A	Chunk B
Token 1 → [……] 384개	Token 1 → [……] 384개
Token 2 → [……] 384개	Token 2 → [……] 384개
Token 3 → [……] 384개	Token 3 → [……] 384개
Token 4 → [……] 384개	…
Token 5 → [……] 384개	Token 8 → [……] 384개
Shape = (5, 384)	Shape = (8, 384)

❖ Sentence Transformer가 pooling한다

Chunk에 Token L개가 있고, embedding dimension이 d

$$H = \begin{bmatrix} h_{11} & h_{12} & \cdots & h_{1d} \\ h_{21} & h_{22} & \cdots & h_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ h_{L1} & h_{L1} & \cdots & h_{Ld} \end{bmatrix} \quad \text{즉,} \quad H \in \mathbb{R}^{L \times d}$$

Mean pooling을 하면 각 dimension별로 평균을 낸다

$$s_j = \frac{1}{L}\sum_{i=1}^{L} h_{ij} \longrightarrow s = [s_1, s_2, \cdots, s_d]$$

$$L \times d \xrightarrow{\text{Mean Pooling}} 1 \times d$$

❖ 그래서 Chunk의 vector가 1 x d 형태로 나타난다

```
embeddings = embedding_model.encode(texts)
```

위 한 줄 안에서 Sentence Transformer가 tokenization –〉 Transformer –〉 Pooling까지 처리

Application method of RAG to LLM

RAG(FAISS의 의미와 활용)

- FAISS의 Code 및 수학적 원리

Sentence Transformer가 각 Chunk를 숫자 Vector로 변환

```
texts = [chunk["text"] for chunk in chunks]

embeddings = embedding_model.encode(
    texts,
    show_progress_bar=True,
    convert_to_numpy=True
)
```

❖ Embedding model을 function f 라고 하면

$$f(\text{text}) = \boldsymbol{x}$$

즉, $\text{Chunk}_1 \rightarrow \boldsymbol{x_1}$

$\text{Chunk}_2 \rightarrow \boldsymbol{x_2}$

$\vdots$

$\text{Chunk}_N \rightarrow \boldsymbol{x_N}$

```
dimension = embeddings.shape[1]
```

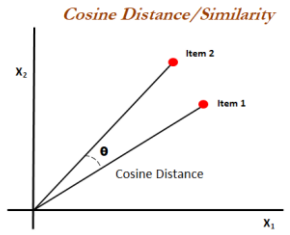
각 Vector 는 $\boldsymbol{x}_i = [x_{i1}, x_{i2}, x_{i3}, \cdots, x_{id}]$

FAISS에 넣기 전에 Normalize

❖ Unit vectorization of a vector

```
faiss.normalize_L2(embeddings)
```

$$\boldsymbol{x} = [x_1, x_2, x_3, \cdots, x_d] \rightarrow \hat{\boldsymbol{x}} = \frac{\boldsymbol{x}}{\|\boldsymbol{x}\|}$$




```
index = faiss.IndexFlatIP(dimension)
```

Flat: Exact Search, FAISS에 저장된 모든 vector를 질문 vector와 직접 비교한다
IP: Inner Product

$$\boldsymbol{q} = [q_1, q_2, q_3 \cdots q_d] \qquad \boldsymbol{x} = [x_1, x_2, x_3 \cdots x_d]$$

$$\text{FAISS Score}(q, x_i) = \frac{q \cdot x_i}{\|q\|\|x_i\|} = cos\theta$$

❖ $cos\theta$ 값으로 방향과 각으로 유사도 판단

Application method of RAG to LLM

RAG(FAISS의 의미와 활용)

- FAISS의 Code 및 수학적 원리

index.add()의 의미

```
index.add(embeddings)
```

❖ index.add는 “이 vector들을 FAISS 검색 대상 database에 등록하겠다“ 라는 의미

❖ Ex)

Chunk 0 → $\boldsymbol{x}_0 = [0.2, 0.7, 0.1, \cdots]$
Chunk 1 → $\boldsymbol{x}_1 = [0.5, 0.3, 0.9, \cdots]$
Chunk 2 → $\boldsymbol{x}_2 = [0.8, 0.1, 0.2, \cdots]$
Chunk 3 → $\boldsymbol{x}_3 = [0.4, 0.6, 0.5, \cdots]$

Add 과정

ID 0 → $\boldsymbol{x}_0$
ID 1 → $\boldsymbol{x}_1$
ID 2 → $\boldsymbol{x}_2$
ID 3 → $\boldsymbol{x}_3$

FAISS가 개념적으로 위처럼 보관

질문 embedding

❖ 문서에서 했던 embedding + normalization을 질문에도 적용

```
question_embedding = embedding_model.encode(
    [question],
    convert_to_numpy=True
).astype("float32")

faiss.normalize_L2(question_embedding)
```

```
scores, indices = index.search(
    question_embedding,
    top_k
)
```

❖ 아래 code는 FAISS 검색의 핵심

Application method of RAG to LLM

RAG(FAISS의 의미와 활용)

- FAISS의 Code 및 수학적 원리

```
scores, indices = index.search(
    question_embedding,
    top_k
)
```

index.search()는 무엇인가

❖ 예로 FAISS 안에 N개의 vector가 있다고 하면

$$x_1, x_2, x_3, \cdots, x_N \qquad \text{query vector는 } q$$

❖ 현재 code에서 IndexFlatIP이므로 FAISS는 개념적으로

$$s_1 = q_1^T x_1$$
$$s_2 = q_2^T x_2$$
$$s_3 = q_3^T x_3$$
$$\vdots$$
$$s_N = q_N^T x_N$$

를 전부 계산한다

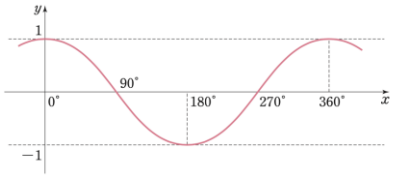
$$\text{FAISS Score}(q, x_i) = \frac{q \cdot x_i}{\|q\|\|x_i\|} = cos\theta$$

❖ Flat index는 검색 시 indexed vector들을 순차적으로 query와 비교하는 exhaustive 방식

❖ Scores, indices는 무엇인가

```
scores =
[[0.921, 0.885, 0.843, 0.798, 0.771]]

indices =
[[25, 104, 7, 62, 311]]
```



-〉 Score는 유사도 점수, indices는 해당 Chunk 번호, 위 예시는 Top-k = 5라고 했을 때 5개의 결과가 나온다

```
chunk = chunks[idx].copy()
chunk["score"] = float(score)
```

❖ FAISS ID -〉 원본 PDF chunk로 돌아간다

❖ Idx = 2라고 하면 `chunk = chunks[2].copy()`

```
{
    "text": "The endurance is approximately 45 minutes.",
    "page": 12
}
```

를 가지고 온다

즉 FAISS는 ID2의 벡터가 질문 벡터와 가장 비슷하다는 것을 판단하고 ID2의 원래 문장을 복원

Application method of RAG to LLM

RAG

- 증강 및 생성

❖ Top-k/p로 선정된 Chunks를 질문 prompt에 추가해서 LLM에게 제공

❖ LLM은 Chunks를 참고해서 답변 생성

질문: 독일로 출장을 갈 때 숙박비 상한선은?
다음 문서를 참고해서 답해줘:
[※ 제10조 제1항 제4호에도 불구하고, 학술회의 등 공식적인 행사에 참석할 경우 개최장소의 숙박시설(호텔 등)을 이용할 때에는 숙박비의 실비(전액)를 정산할 수 있다.]
[1. 국가 및 도시별 등급구분은 다음과 같다. 나등급: 유럽주]
[교수, 부교수 책임급: 숙박비 상한- 가) 155달러, 나) 123 달러, 다) 90 달러]

RAG에 의해 증강된 prompt

❖ 모두 외부 '문서'를 LLM이 참고해서 답변을 하는 상황으로 RAG 설명

❖ 외부 '문서'가 인터넷 즉, 모든 온라인 정보라면 '검색'을 수행하여 얻어낸 정보 활용

❖ Model은 Update가 어렵지만 prompt는 얼마든지 검색을 통해 실시간 Update 가능

Application method of RAG to LLM

Intelligent Design for Engineering Applications Lab

RAG Code의 구현 방식

❖ 현재 코드에(RAG만 있는 경우) 대응시켰을 때:

Retrieval
➔ FAISS로 관련 문서 검색

Augmented
➔ 검색된 문서를 Prompt에 추가

Generation
➔ Mistral이 Prompt를 보고 답변 생성

❖ RAG라는 특별한 하나의 함수가 따로 존재하는 것이 아니라 하나의 로직 과정

```
# 1. Retrieval
retrieved_docs = retrieve_documents(question, ...)

# 2. Augmentation
context = 검색된 문서들을 하나로 합침

# 3. Prompt
prompt = context + question

# 4. Generation
answer = ask_mistral(prompt)
```

Code분석

❖ Ask_with_rag()

```
def ask_with_rag(question, embedding_model, index, chunks, top_k=5):

    # 1. Retrieval
    retrieved_docs = retrieve_documents(
        question,
        embedding_model,
        index,
        chunks,
        top_k
    )

    # 2. Context
    context = "\n".join(
        doc["text"]
        for doc in retrieved_docs
    )

    # 3. Augmented Prompt
    prompt = f"""
    Context:
    {context}

    Question:
    {question}

    Answer:
    """

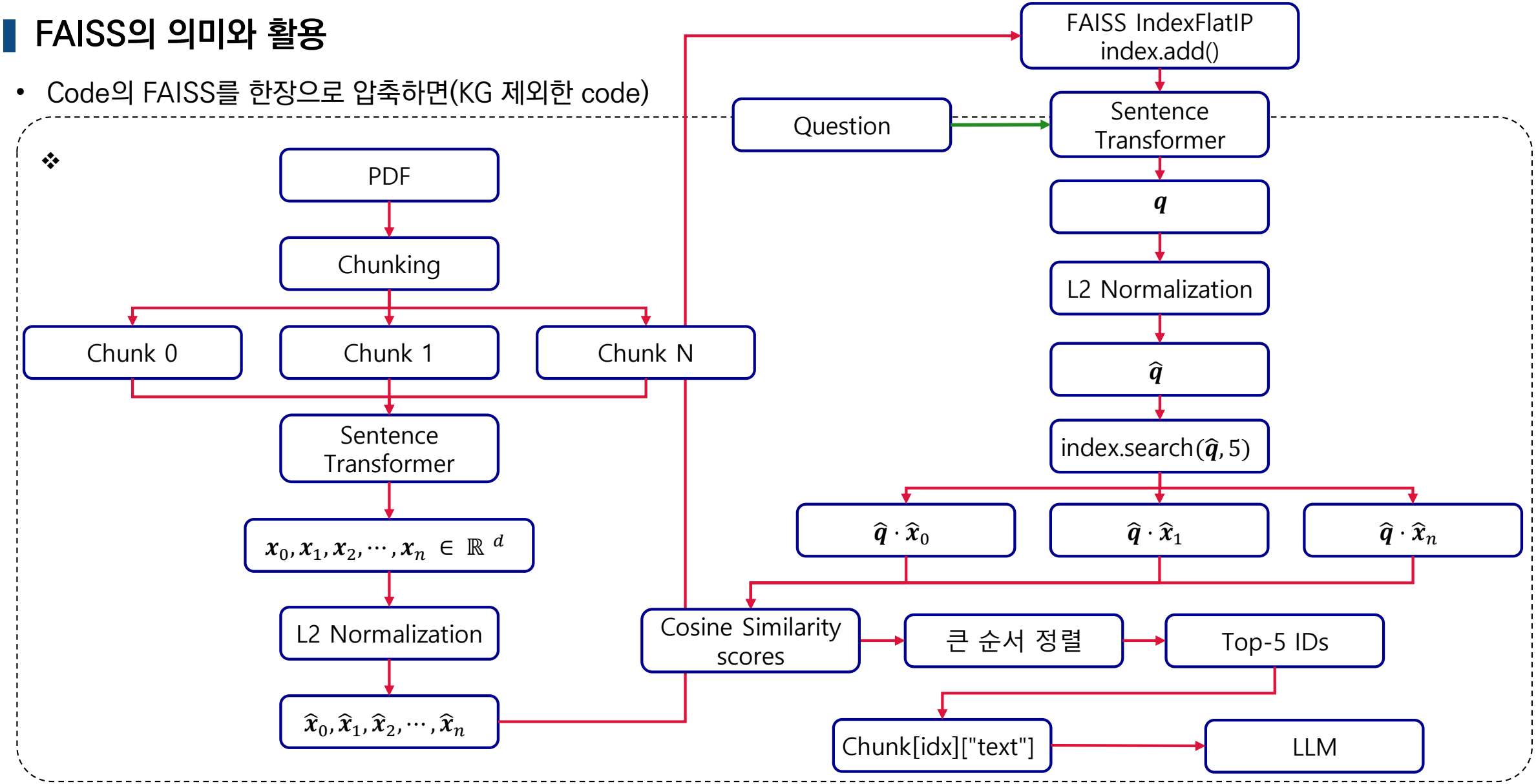
    # 4. Generation
    answer = ask_mistral(prompt)

    return answer
```

Application method of RAG to LLM

FAISS의 의미와 활용

- Code의 FAISS를 한장으로 압축하면(KG 제외한 code)



Application method of RAG to LLM

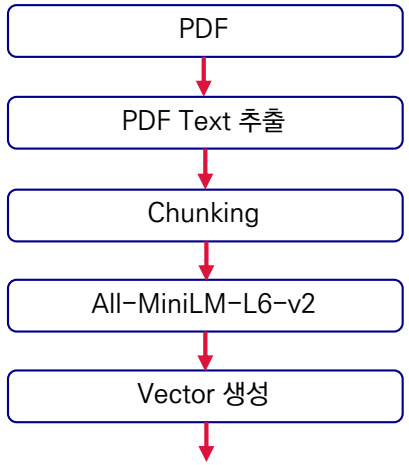
NoRAG vs RAG Test

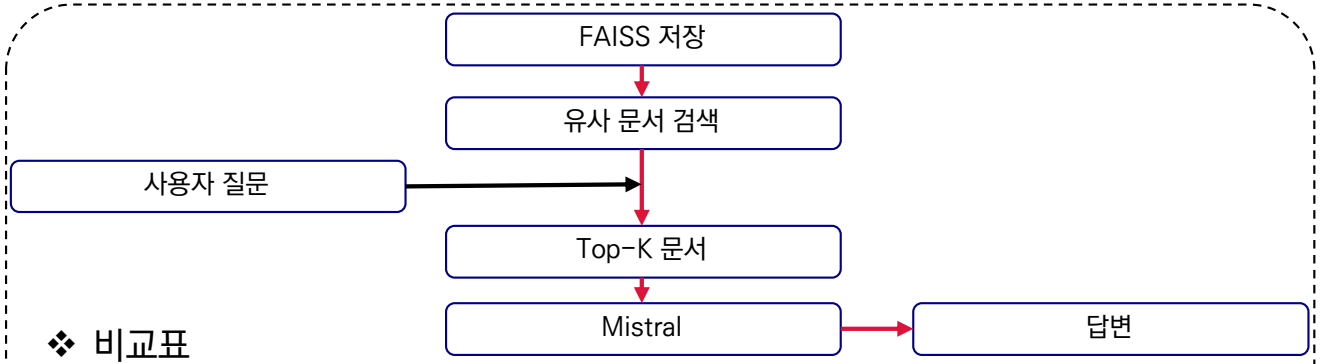
- 실험에 사용한 자료
- NoRAG와 RAG의 비교

❖ Pdf 제목: The Drone Databook

❖ 핵심 내용: 각 국가가 군사적, 안보적 목적으로 어떤 드론을 보유하고, 어떤 부대가 운용하며, 어디에서 운용하고, 실제 작전에 어떻게 사용했으며, 어떤 드론을 개발하고 수출하는가

❖ Python을 활용하여 코딩





❖ 비교표

Q	질문 유형	No-RAG	RAG	평가
1	문서 구조	✗ 잘못된 일반론	☑ 정확	RAG 우수
2	Inventory 설명	✗ 일반적인 추측	☑ 대체로 정확	RAG 우수
3	용어 정의	✗ 일반적인 의미로 설명	☑ 정확	RAG 우수
4	Infarstructure 설명	✗ 잘못된 의미	☑ 정확	RAG 우수
5	특정 국가	✗ 사실과 다른 내용 다수	⚠ 일부 정확	RAG 우수하지만 불완전
6	특정 국가 Inventory	✗ 정보 없음	⚠ 정보를 찾았지만 답변이 불완전	RAG 우수
7	특정 UAV	✗ 환각	✗ 검색 실패	둘 다 실패
8	기술 제원	✗ 일반론	✗ 다른 section과 혼동	RAG 실패

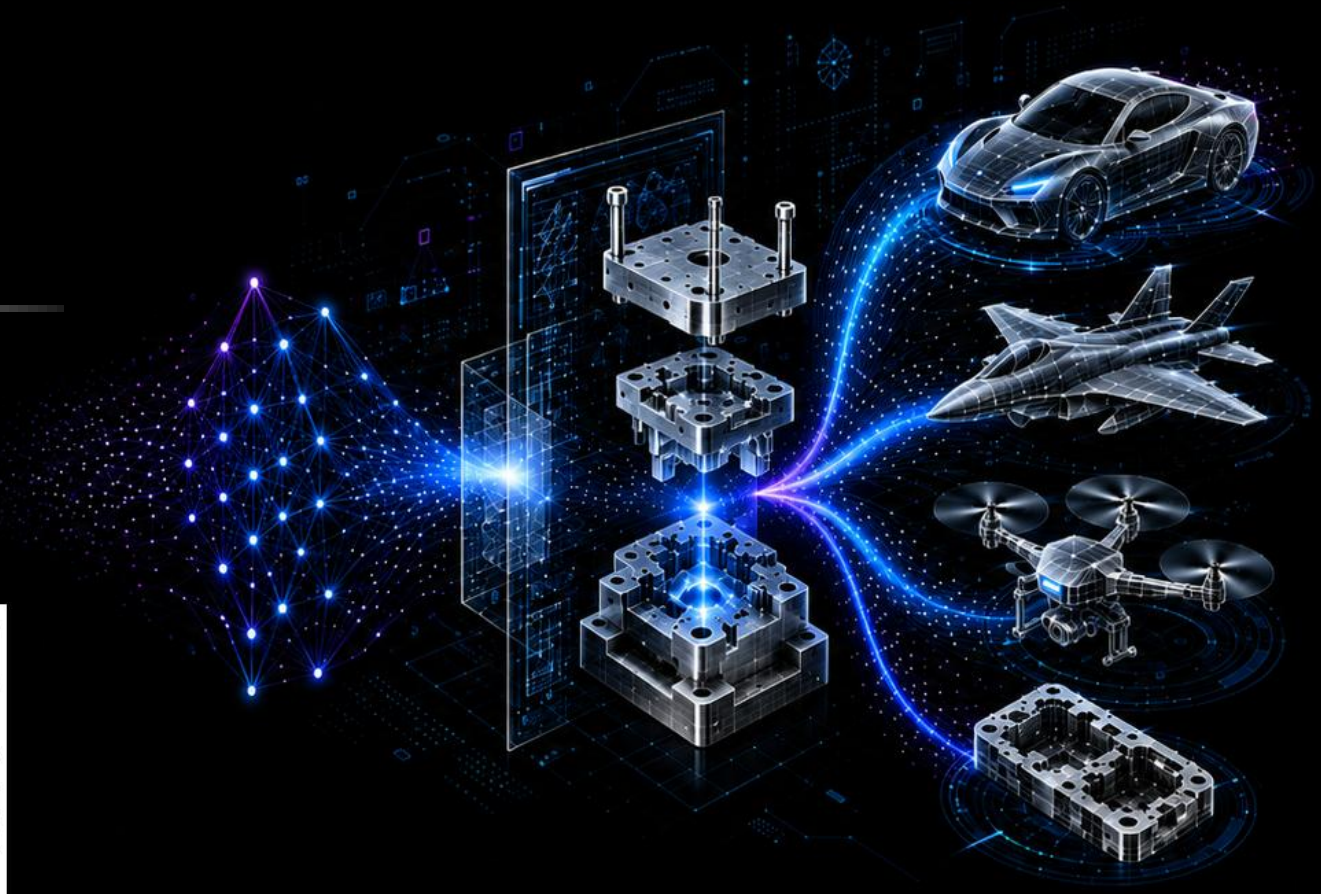
Any Questions?

이름 신 동 우

소속 한양대학교 기계공학과

이메일 sunwoongy@hanyang.ac.kr

Intelligent Design for Engineering Applications Lab

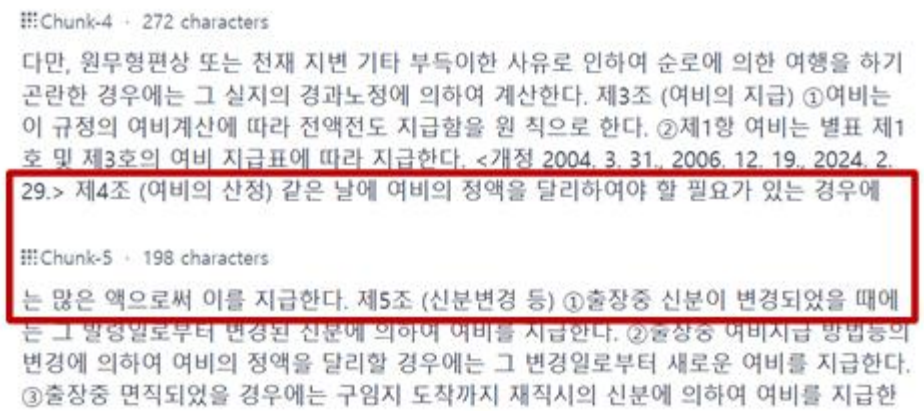


Application method of RAG to LLM

Intelligent Design for Engineering Applications Lab

문서 기반의 RAG의 한계

- Chunking 과정에서 관련 문맥이 분리될 수 있음
- 정보가 여러 문서에 흩어져 있을 수 있음
- 문장 사이의 연결 관계, 인과 방향, 그리고 개념 간 상하 관계 표현 불가



제15조 (일비등의 감액) ①같은 곳에 장기간 체재하는 경우의 일비는 도착한 다음 날부터 기산하여 15일을 초과한 때에는 그 초과일수에 대하여 정액의 1할, 30일을 초과한 때에는 그 초과일수에 대하여 정액의 2할, 60일 초과한 때에는 그 초과일수에 대하여는 정액의 3할에 상당한 액을 감액지급한다. <개정 2004. 3. 31., 2006. 4. 19., 2022. 3. 24.>

②같은 곳에 장기간 체재하는 경우의 숙박비는 30일까지는 전액을 지급하되, 30일 초과한 때에는 그 초과일수에 대하여 제1항을 준용한다. <신설 2022. 3. 24.> <기존 제2항은 제3항으로 이동 2022. 3. 24.>

Application method of RAG to LLM

Knowledge Graph

- Knowledge Graph가 무엇인가
- Knowledge Graph 이해 그림

❖ Knowledge Graph(KG): 지식을 단순히 저장하는 것이 아닌 관계를 통해 맥락을 명시하여 사용하는 그래프

❖ KG의 기본정의: Entity, Relation, Fact의 집합으로 정의

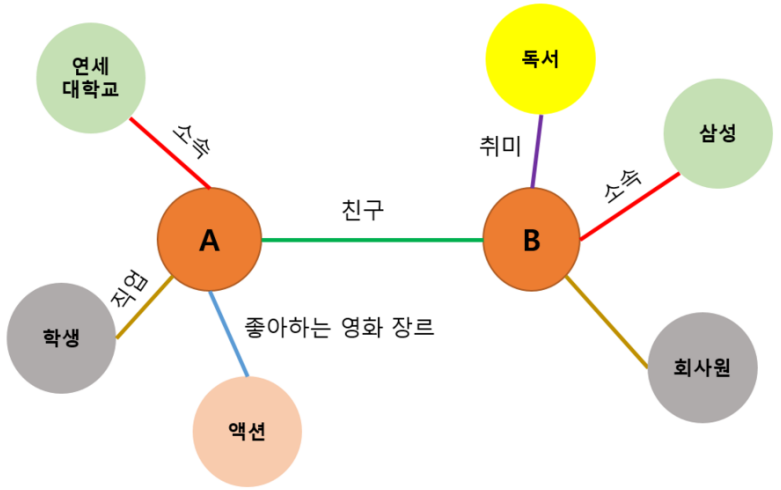
❖ Entity: 무엇에 대한 이야기인가?

❖ Relation: 그것들을 어떻게 연결되는가?

❖ Fact: 그 연결에 대해 실제로 성립하는 구체적인 정보

❖ A, B는 Entity(Node)

❖ A, B 사이의 친구는 Relation(Edge)



Application method of RAG to LLM

Knowledge Graph와 Ontology 차이점

- Ontology
- 차이점

❖ 정의: 어떤 도메인의 Concept과 Relationship을 정형화해서 정의한 이론적 모델

❖ 초점: “어떤 개념들이 존재하고, 어떻게 정의되어야 하는가?”

❖ 구성 요소:

Class: 개념 범주(ex: Character, Organization, Location)

Property: 관계의 규칙(ex: Character → member_of → Organization)

Axioms: 논리 제약 조건(ex: 모든 Character는 반드시 하나의 Organization에 소속될 수 있다.)

❖ RDF, OWL 기반의 도메인 모델 → “애니메이션 세계관 Ontology”를 정의해 두면, 어떤 작품이든 같은 구조로 지식 그래프를 만들 수 있다.

❖ 특징: 지식 그래프의 **설계도/청사진** 같은 역할

RDF(Resource Description Framework)
→ data를 주체(subject), 술어(Predicate), 객체(Object)의 삼중 구조로 표현하여 data 간의 관계를 명확히 나타내는 방법

❖ 차이점

구분	Knowledge Graph	Ontology
초점	실제 데이터/사실 (instances)	개념, 관계의 정의 (schema)
표현	그래프 데이터베이스 (node, edge)	논리 모델(RDF, OWL 등)
목적	지식 연결, 탐색, 추론	지식 구조 정의, 일관성 보장
비유	“완성된 지도 "	“지도 그리는 설계도 "

OWL(Web Ontology Language)
→ RDF가 기본적인 data 관계를 표현한다면, OWL은 그 위에 복잡한 논리적 구조와 제약 조건을 추가로 표현할 수 있게 한다.

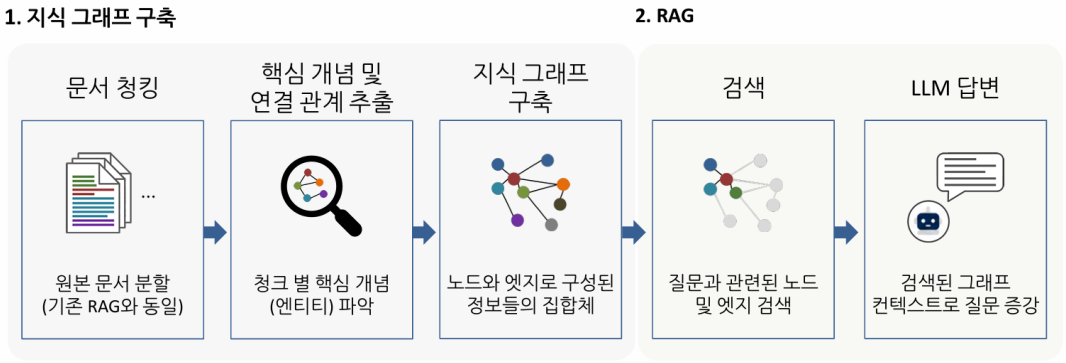
Application method of RAG to LLM

Knowledge Graph의 활용

- 관계 기반 검색을 통해 증강
- Graph RAG = 지식 그래프 + (RAG: 검색 + 증강 + LLM 생성)

Graph RAG

❖



관련 논문

❖ 논문 제목: Knowledge graph–extended retrieval augmented generation for question answering, 2025, Applied Intelligence

❖ 논문의 핵심적인 실험적 결론: 질문이 복잡해질수록 Question Decomposition + KG retrieval의 조합이 유리하다

❖ LLM을 학습시키지 않고, KG의 정확한 구조적 사실 검색과 LLM의 자연어 이해 능력을 Question Decomposition으로 연결한 KG-RAG 구조를 제안

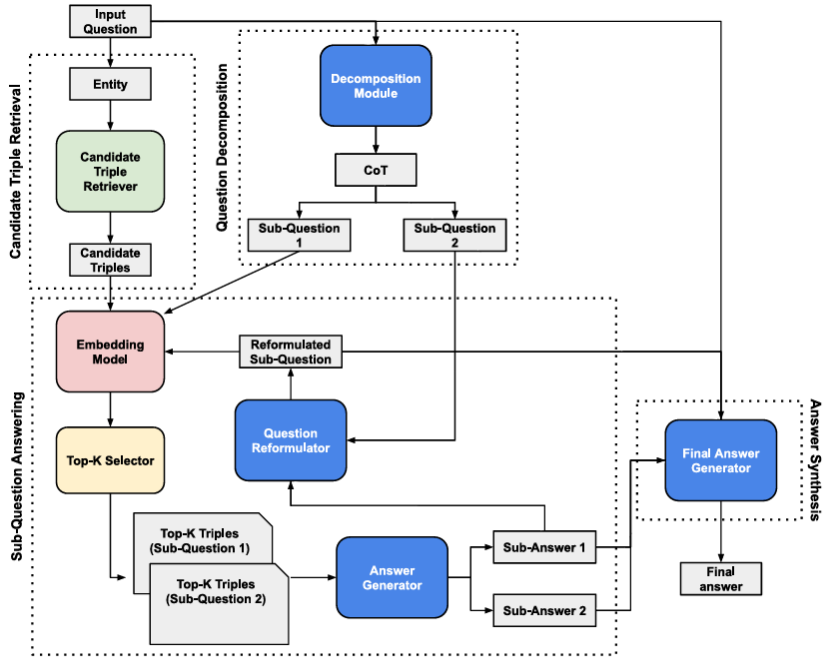


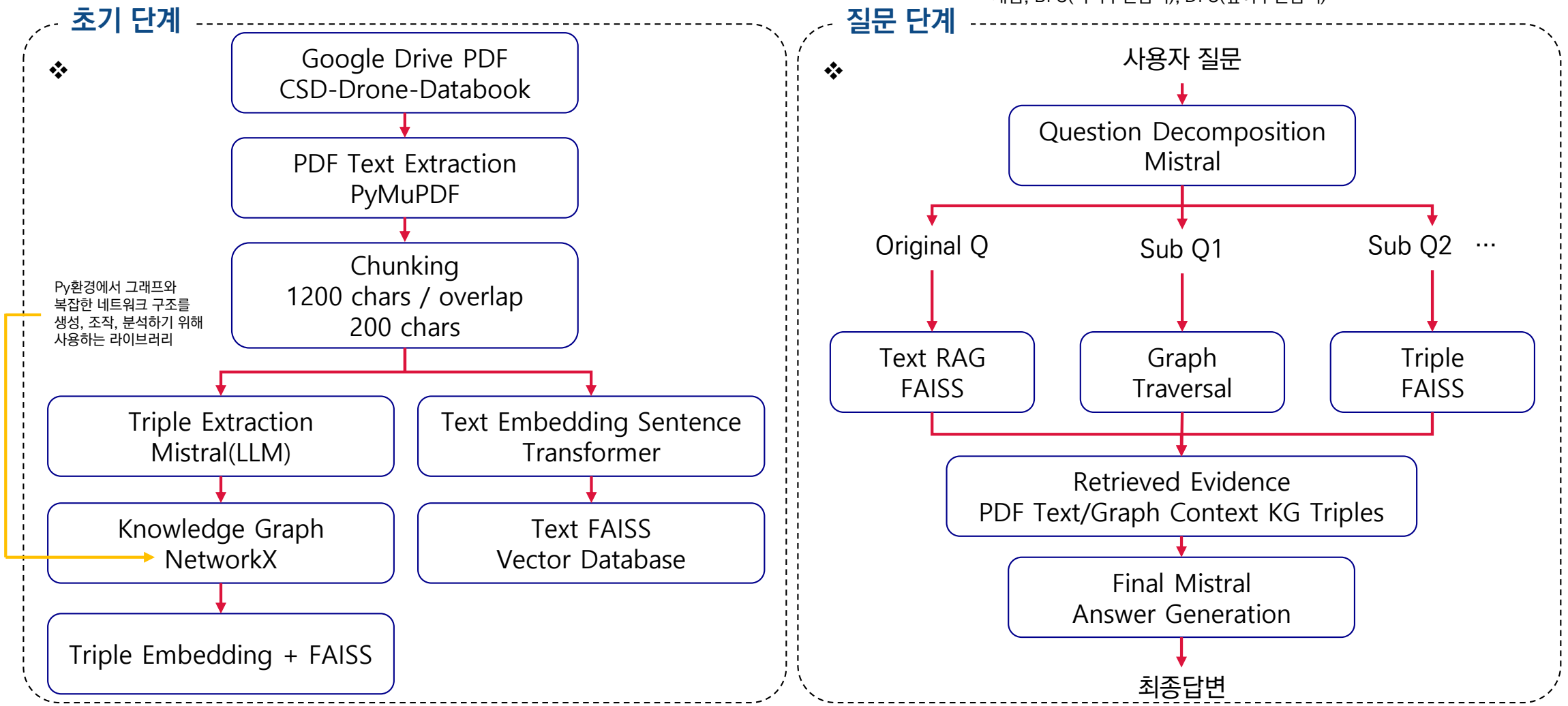
Fig. 1 The architecture of the proposed system. An example of a 2-hop question is included, to give an idea of the data structures that are involved in the end-to-end process. The green color indicates processing with the KG; the red block shows the embedding model and the blue modules utilize an LLM

Application method of RAG to LLM

Knowledge Graph의 활용

- Knowledge Graph의 활용 code 구현(Colab 기준)

Graph Traversal(순회): 그래프처럼 연결된 구조에서 Node를 한 번씩 탐색하는 개념, BFS(너비우선탐색), DFS(깊이우선탐색)



Application method of RAG to LLM

Knowledge Graph의 활용

- Extracting triples 11/841은 무엇인가

❖ Triples = Subject(주어) + Predicate(관계) + Object(목적어)

✓ 하나의 사실(fact)을 표현하는 최소 단위

1. Chunk 11
2. Mistral에게 전달
3. "이 텍스트에서 사실 관계를 찾아라"
4. (subject, relation, object)
5. Triple
6. Knowledge Graph에 추가

Code 중간 과정

❖ 814개의 chunk중 현재 9, 10번째 chunk에서 KG용 triple을 추출 중

[transformers] Both `max_new_tokens` (=512) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
Extracting triples 9/841
[transformers] Both `max_new_tokens` (=512) and `max_length`(=20) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
Extracting triples 10/841

Application method of RAG to LLM

Knowledge Graph의 활용

- Code 결과 분석

질문 유형

1. What section are included in the country profiles?
2. What information is included in the inventory section?
3. What is the difference between Dedicated, Partial, and Not Dedicated drone units?
4. What drones are listed in New Zealand's active inventory?
5. Which countries operate the Barmor UAV?
6. Which UAVs are manufactured by C-Astral?
7. What military units operate drones in Australia?
8. Which countries are connected to Bramor through exports or operation?

전체 실행 결과

❖ Code 실행 총 시간: 약 2시간 이상

평가 항목	결과	평가
PDF Chunk	841개	☑ 정상
Triple	11,029개	☑ 정상
Chunk 처리 실패	3개	▯ 매우 적음
Chunk 처리 성공률	99.64%	▯ 매우 높음
Text FAISS	생성 완료	☑
Triple FAISS	생성 완료	☑
KG Retrieval	수행됨	▯
질문 처리 완료	8개	▯
미완료 질문	나머지 질문	▩
답변 생성	8개 모두 생성	▯

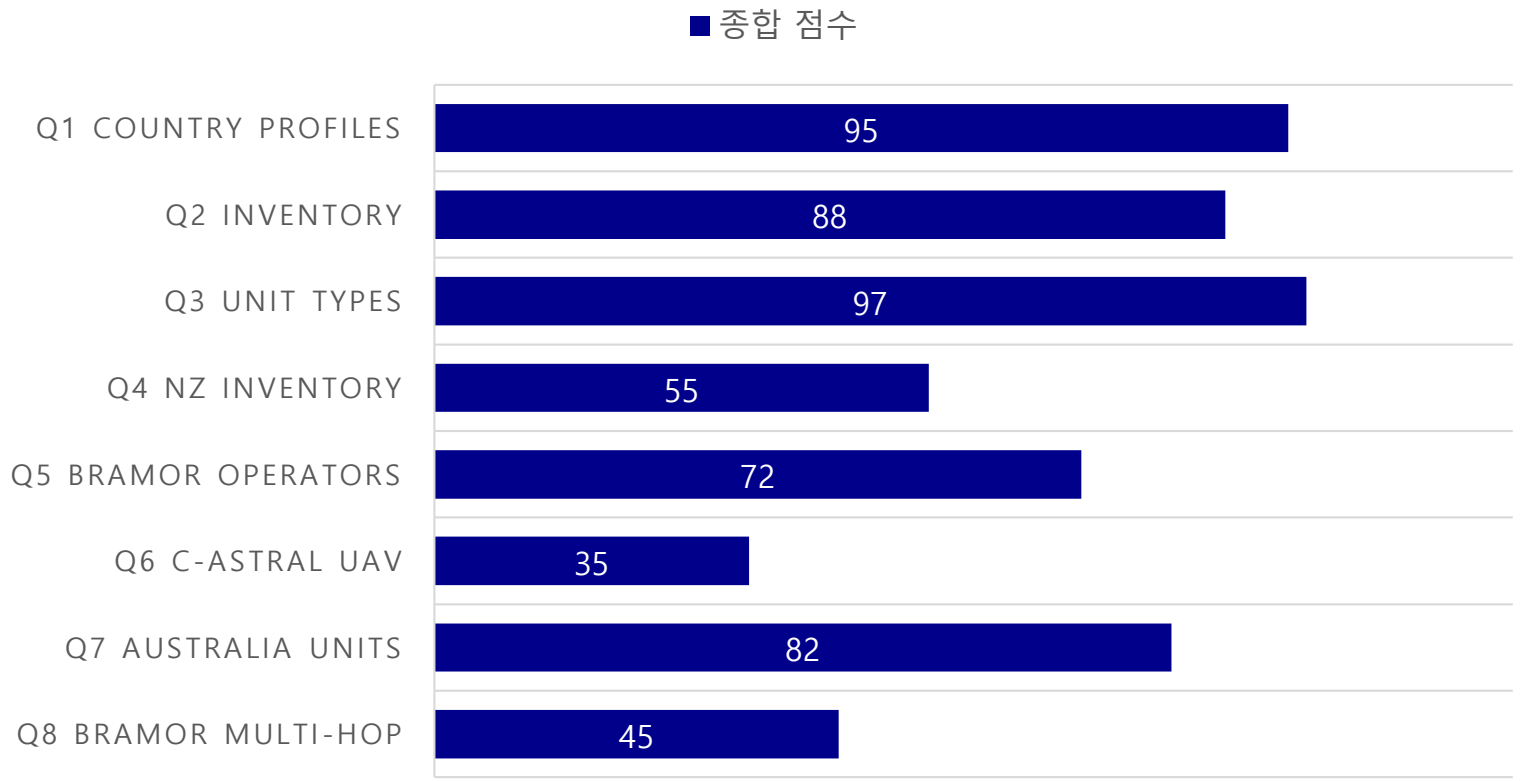
Application method of RAG to LLM

Knowledge Graph의 활용

- Code 결과 분석

질문별 최종 답변 품질 평가

❖



점수	의미
90~100	거의 정확, 질문에 직접 답변
80~89	정확하지만 일부 불필요한 내용 존재
70~79	핵심 답변은 가능하지만 일부 오류/ Noise
60~69	부분적으로 맞지만 중요한 문제가 있음
40~59	관련 정보는 있으나 질문에 제대로 답하지 못함
0~39	사실상 실패

Manual Evaluation, 90점 이상: 매우 우수, 80~89: 우수, 70~79: 부분 성공, 60~69: 미흡, 60미만: 실패

Application method of RAG to LLM

Knowledge Graph의 활용

- Code 결과 분석

❖ Graph Retrieval이 성공했다고 좋은 답변이 나오는가?

Q1
➔ Graph = 8
➔ Answer = 95

Q3
➔ Graph = 21
➔ Answer = 97

Q4
➔ Graph = 155
➔ Answer = 55

Q8
➔ Graph = 681
➔ Answer = 45

1. Graph를 많이 가져올수록 반드시 좋은 답변이 나오는 것은 아니다.
2. 오히려 Q4, Q8에서는 Graph Context가 너무 많아져서 Noise가 증가했을 가능성이 큼

현재 시스템의 핵심 문제 3가지

❖ KG가 존재하지만 항상 질문과 관련된 정보를 주지는 않는다, Q6

C-Astral → Manufactures → Bramor UAV

LLM이 문맥을 잘못 이해하고 사용할 수 없는 Triple 생성

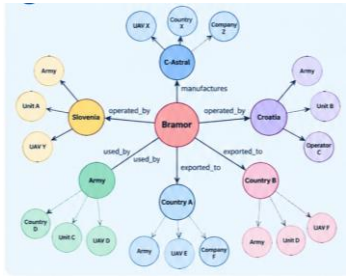
C-Astral → Manufactures → Spaceplane

질문: C-Astral이 제조하는 UAV는?

-〉 Triple 자체의 품질이 중요

❖ Graph Search가 너무 많은 정보를 가져온다, Q8

Bramor → 1-hop → 주변 Entity → 2-hop → 그 주변 Entity



MAX_HOPS = 2

❖ Final Answer가 Retrieval 결과를 충분히 필터링하지 못한다, Q4

Text Retrieval + Triple Retrieval + Graph Retrieval + Question → Mistral → Final Answer

질문: What drones are listed in New Zealand's active inventory?

New Zealand → Active inventory → Drone 목록

LLM이 Question: "What drones?"를 "What are the specifications of these drones?" 처럼 해석해 버린 것

검색 결과에는 Drone의 specification이나 다른 관련 정보가 섞여 있기 때문

Application method of RAG to LLM

Knowledge Graph의 활용

- Code 결과 분석

결론(정리)

❖ PDF로부터 841개의 Chunk와 11,029개의 Triple을 성공적으로 구축하여 KG-RAG의 전체 파이프라인 정상적으로 구현

❖ 841개 Chunk 중 838개가 정상 처리되어 KG 구축 단계의 처리 성공률은 약 99.64%

❖ 8개의 test 질문 중에서 4개는 높은 품질로 답변 되었다. -> 질문 표본이 너무 적다

❖ Bramor 및 C-Astral과 같은 특정 Entity 질문에서는 KG에 적절한 관계가 형성되지 않아 Graph Retrieval이 실패하거나 부정한 Triple이 검색되는 문제

❖ Multi-hop 질문에서는 Graph Retrieval 결과가 최대 681개까지 증가하면서 관련 없는 정보가 최종 답변에 포함되는 문제 발생

Any Questions?

이름 신 동 우

소속 한양대학교 기계공학과

이메일 sunwoongy@hanyang.ac.kr

Intelligent Design for Engineering Applications Lab

